

bok25

基
礎
班

、**考研数据結構**



图

概念多

算法多

考点多

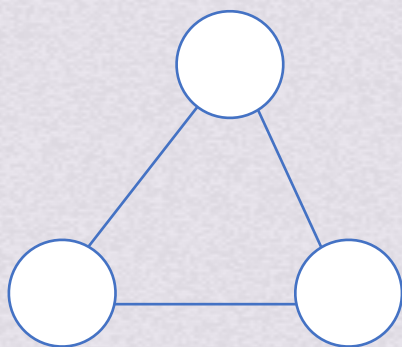


图的定义

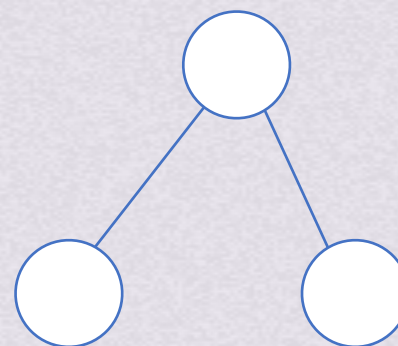
$$G = (V, E)$$

$V \rightarrow$ Vertex, 顶点的有穷非空集合 $|V| \rightarrow$ 顶点数

$E \rightarrow$ Edge, 边的有穷集合, 可以为空 $|E| \rightarrow$ 边数



图



既是图，又是树

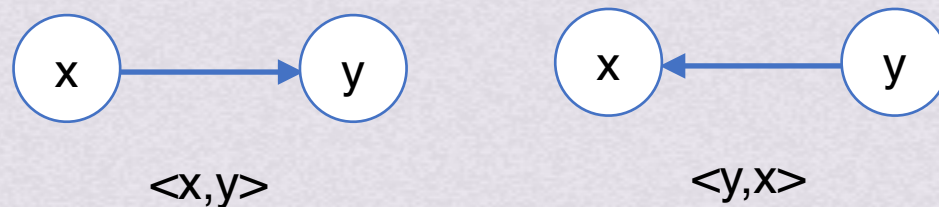


图的术语

无向图：顶点 x 和 y 之间的边是无序的， (x, y) 和 (y, x) 指的是同一条边



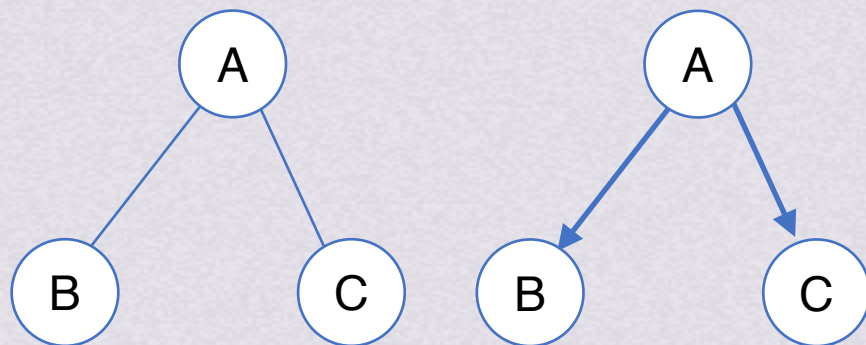
有向图：顶点 x 和 y 之间的边是有序的， $\langle x, y \rangle$ 和 $\langle y, x \rangle$ 指的是两条不同的边





度

度：顶点 v 的度指的是的和顶点 v 相关联的边的数目，记为 $TD(v)$

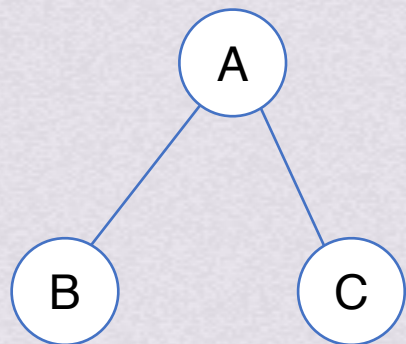


A的度为2
B的度为1
C的度为1



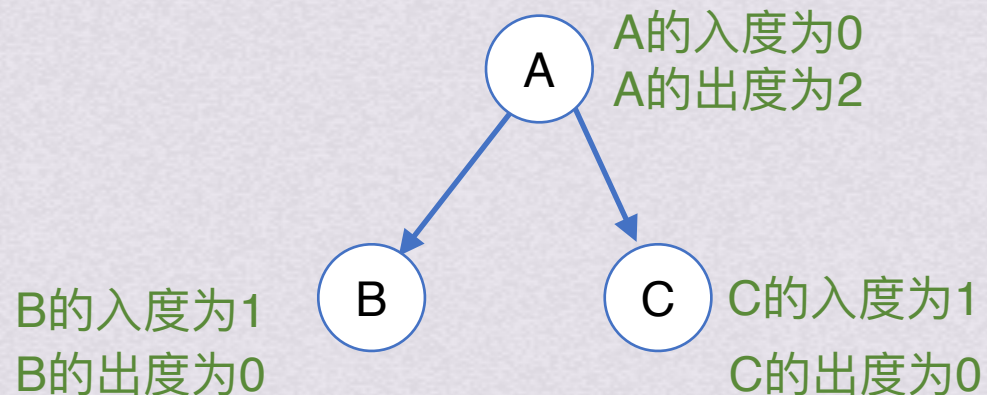
度

度：顶点 v 的度指的是的和顶点 v 相关联的边的数目，记为 $TD(v)$



入度：以顶点 v 为弧头的弧的数目，记为 $ID(v)$

出度：以顶点 v 为弧尾的弧的数目，记为 $OD(v)$

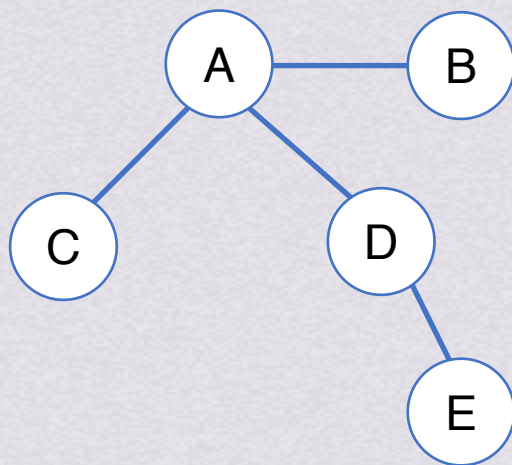


$$\text{度} = \text{入度} + \text{出度}$$
$$TD(v) = ID(v) + OD(v)$$

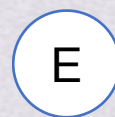
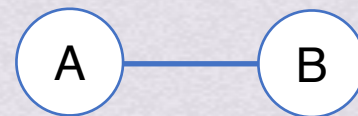


子图：假设有两个图 $G = (V, E)$ 和 $G' = (V', E')$ ，如果 $V' \subseteq V$ ， $E' \subseteq E$ ，则称 G' 为 G 的子图

判断以下图是否是 G 的子图



图G

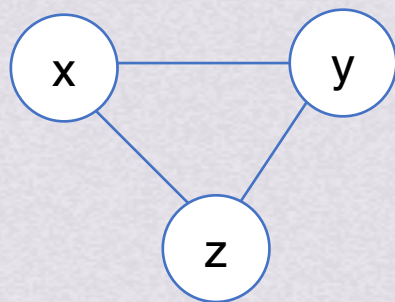




子图：假设有两个图 $G = (V, E)$ 和 $G' = (V', E')$ ，如果 $V' \subseteq V$ ， $E' \subseteq E$ ，则称 G' 为 G 的子图。

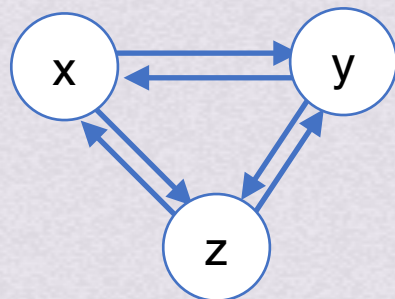
完全图：让图中的边数达到最大，使顶点拥有最多的边

无向图：若具有 $\frac{n(n-1)}{2}$ 条边，则称为无向完全图



具有 $\frac{3(3-1)}{2} = 3$ 条边的无向完全图

有向图：若具有 $n(n-1)$ 条边，则称为有向完全图



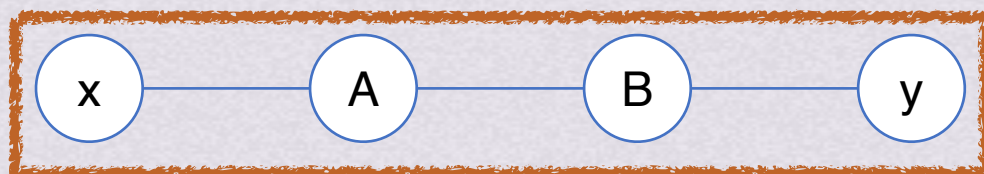
具有 $3(3-1)=6$ 条边的有向完全图



连通、连通图和连通分量：

连通：无向图G中，如果从顶点x到顶点y有路径，则称x和y是连通的

【注】有路径不一定是直接有边

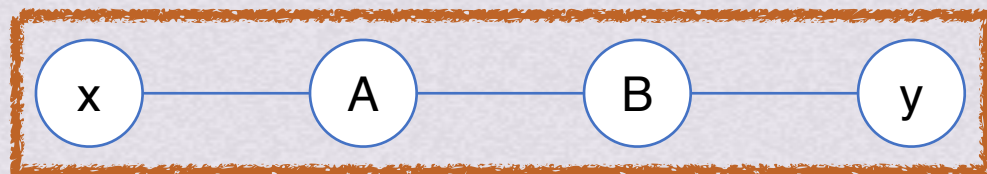


该图的唯一一个连通分量

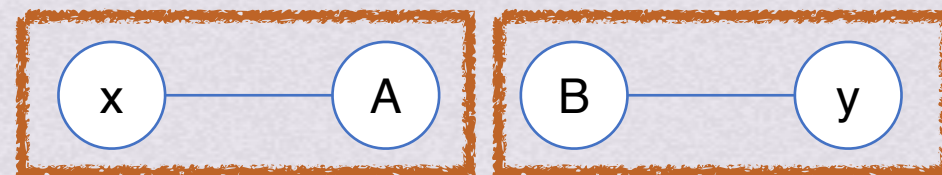
x与y连通

连通图：无向图G中，如果任意两个顶点都是连通的，则称G是连通图

【注】只有单个顶点的图也属于连通图。



连通图 该图的唯一一个连通分量



非连通图
该图的2个连通分量

连通分量：无向图G中的极大连通子图称为图G的连通分量



连通、连通图和连通分量：

连通分量：无向图G中的极大连通子图称为图G的连通分量。

极大连通子图：

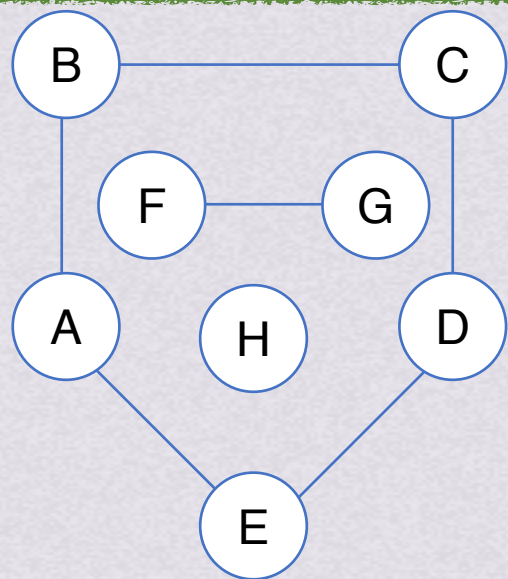
拥有原图的部分或全部顶点。

顶点数固定时，边数达到顶点能承受的最大数量
能连通的点边都放在一个连通子图中，使它足够大，以
至于再多包含一个点或边它变成不连通的了

极小连通子图：

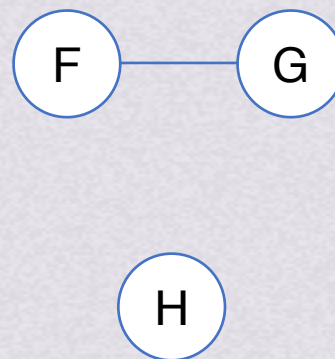
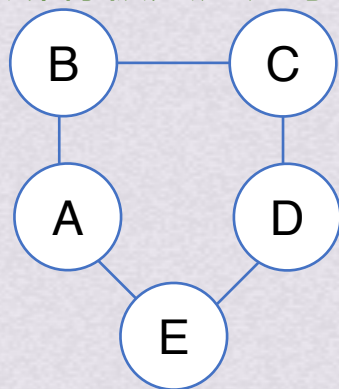
拥有原图的部分或全部顶点。

在保证连通的情况下，边数可以达到的最小数量

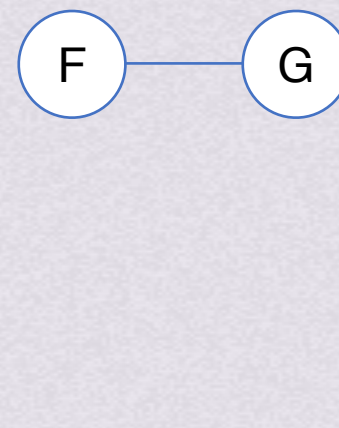
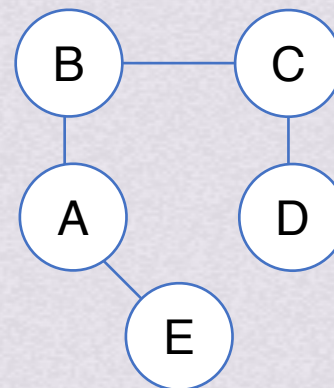


图

所有极大连通子图



部分极小连通子图





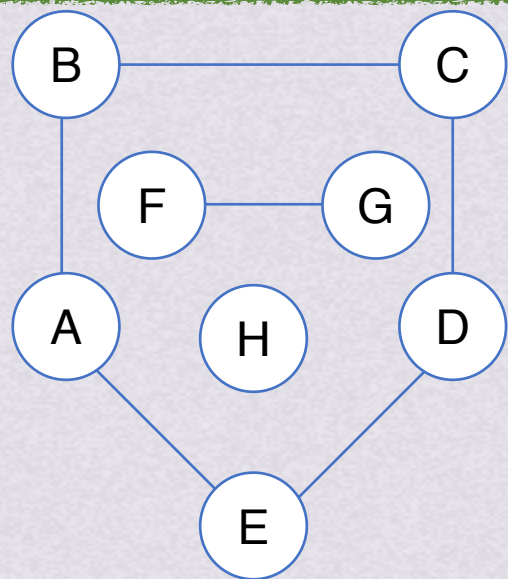
连通、连通图和连通分量：

连通分量：无向图G中的极大连通子图称为图G的连通分量。

极大连通子图：

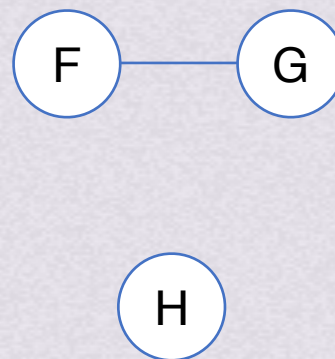
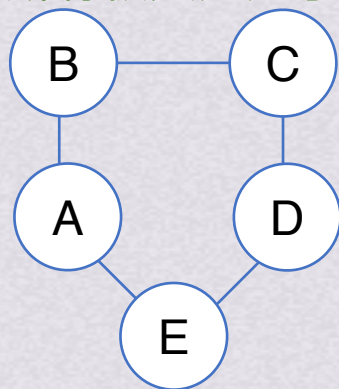
拥有原图的部分或全部顶点。

顶点数固定时，边数达到顶点能承受的最大数量
能连通的点边都放在一个连通子图中，使它足够大，以
至于再多包含一个点或边它变成不连通的了



图

所有极大连通子图



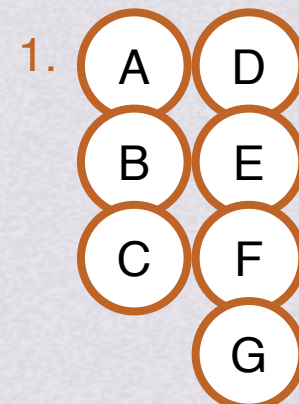
极小连通子图：

拥有原图的部分或全部顶点。

在保证连通的情况下，边数可以达到的最小数量

常见错误

错因：极大连通子图应保证和顶点有关的边都加上



(除H外的单个节点)
是极大连通子图



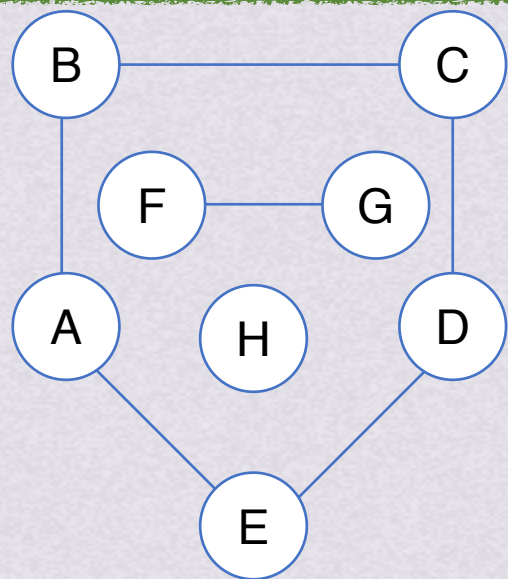
连通、连通图和连通分量：

连通分量：无向图G中的极大连通子图称为图G的连通分量。

极大连通子图：

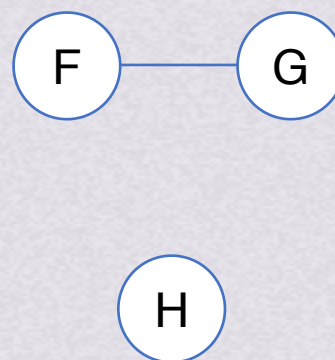
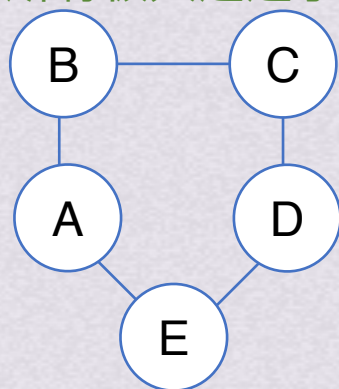
拥有原图的部分或全部顶点。

顶点数固定时，边数达到顶点能承受的最大数量
能连通的点边都放在一个连通子图中，使它足够大，以
至于再多包含一个点或边它变成不连通的了



图

所有极大连通子图

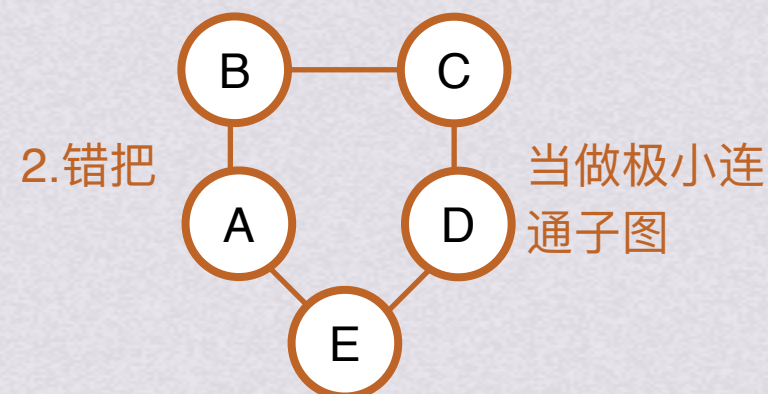


极小连通子图：

拥有原图的部分或全部顶点。

在保证连通的情况下，边数可以达到的最小数量

常见错误



错因：边数没有减少到最小。
该图再去掉一个边仍可保证连通



连通、连通图和连通分量：

连通分量：无向图G中的极大连通子图称为图G的连通分量。

极大连通子图：

拥有原图的部分或全部顶点。

顶点数固定时，边数达到顶点能承受的最大数量
能连通的点边都放在一个连通子图中，使它足够大，以
至于再多包含一个点 它就变成不连通的了

极小连通子图：

拥有原图的部分或全部顶点。

在保证连通的情况下，边数可以达到的最小数量



连通、连通图和连通分量：

连通分量：无向图G中的极大连通子图称为图G的连通分量。

极大连通子图：

拥有原图的部分或全部顶点。

顶点数固定时，边数达到顶点能承受的最大数量
能连通的点边都放在一个连通子图中，使它足够大，以
至于再多包含一个点，它就变成不连通的了

极小连通子图：

拥有原图的部分或全部顶点。

在保证连通的情况下，边数可以达到的最小数量

争议QA

Q.单独的点是否连通？

A.图论中，单独的顶点会被认为是连通的，因为不存在需要连接的其他顶点

Q.从图中摘出单独的点，该子图是否属于极小连通子图？

A.一个单独的顶点可以被认为是一种特殊情况下的连通子图，但它不符合“极小连通子图”的常规定义，因为后者强调的是通过边的最小化选择保持子图的连通性。

在更一般的讨论中，当我们讨论极小连通子图时，我们通常是指包含两个或更多顶点的子图，这些顶点通过边以某种最小的方式连接，保持了子图的连通性

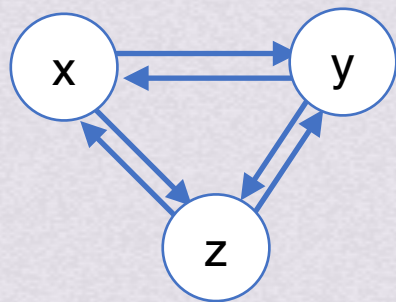
*注：这里所有关于连通的讨论，对象都是无向图



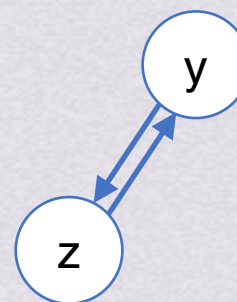
强连通图和强连通分量

强连通图：有向图G中，如果每一对顶点之间都存在路径，则称G是强连通图

【注】由于弧是有方向的，所以每两个顶点必须有反向的两条路径才能称为强连通。



一个有三个顶点的强连通图



y与z强连通



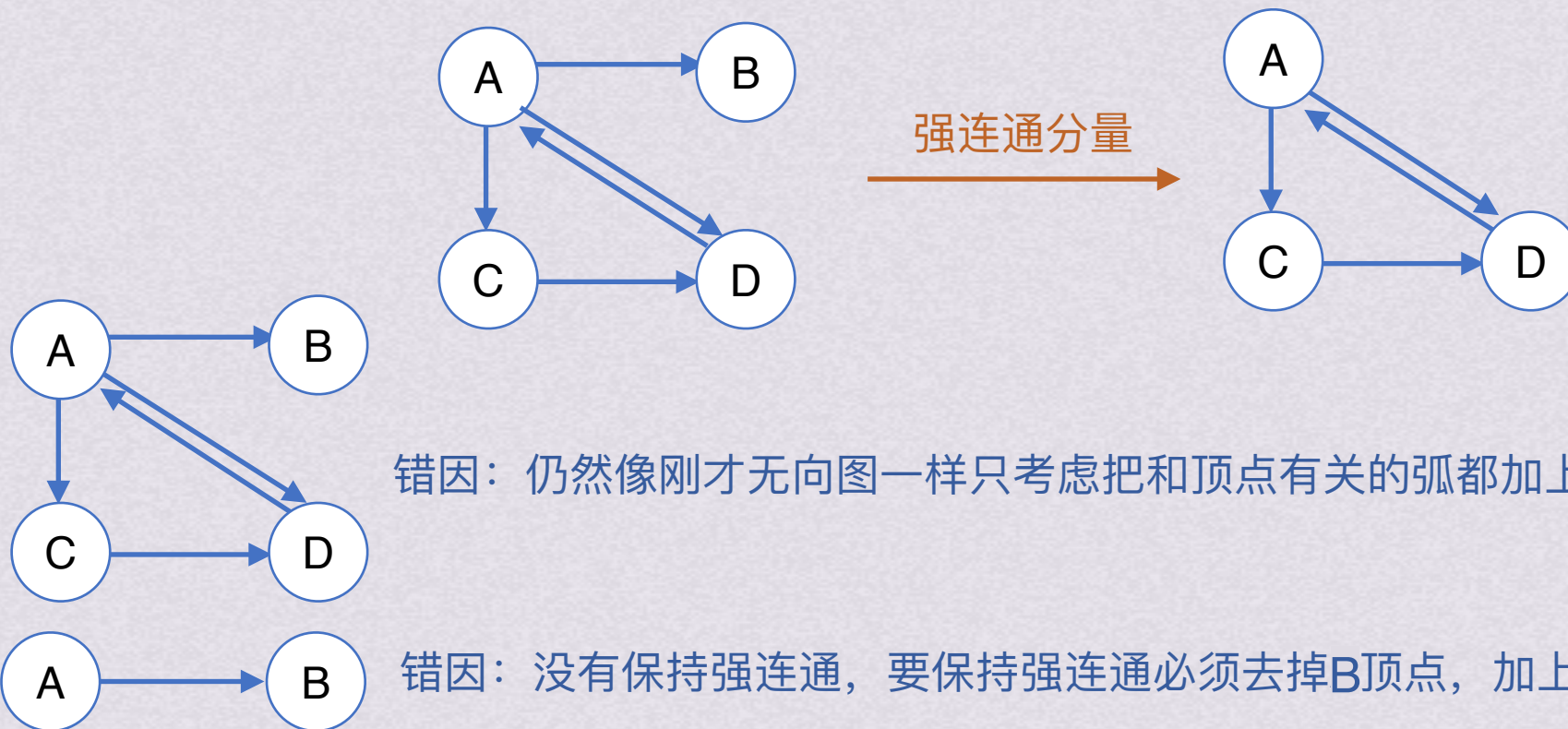
强连通图和强连通分量

强连通图：有向图G中，如果每一对顶点之间都存在路径，则称G是强连通图。

【注】由于弧是有方向的，所以每两个顶点必须有反向的两条路径才能称为强连通。

强连通分量：有向图G中的极大强连通子图称为图G的强连通分量。

【注】不是强连通图也可以有强连通分量



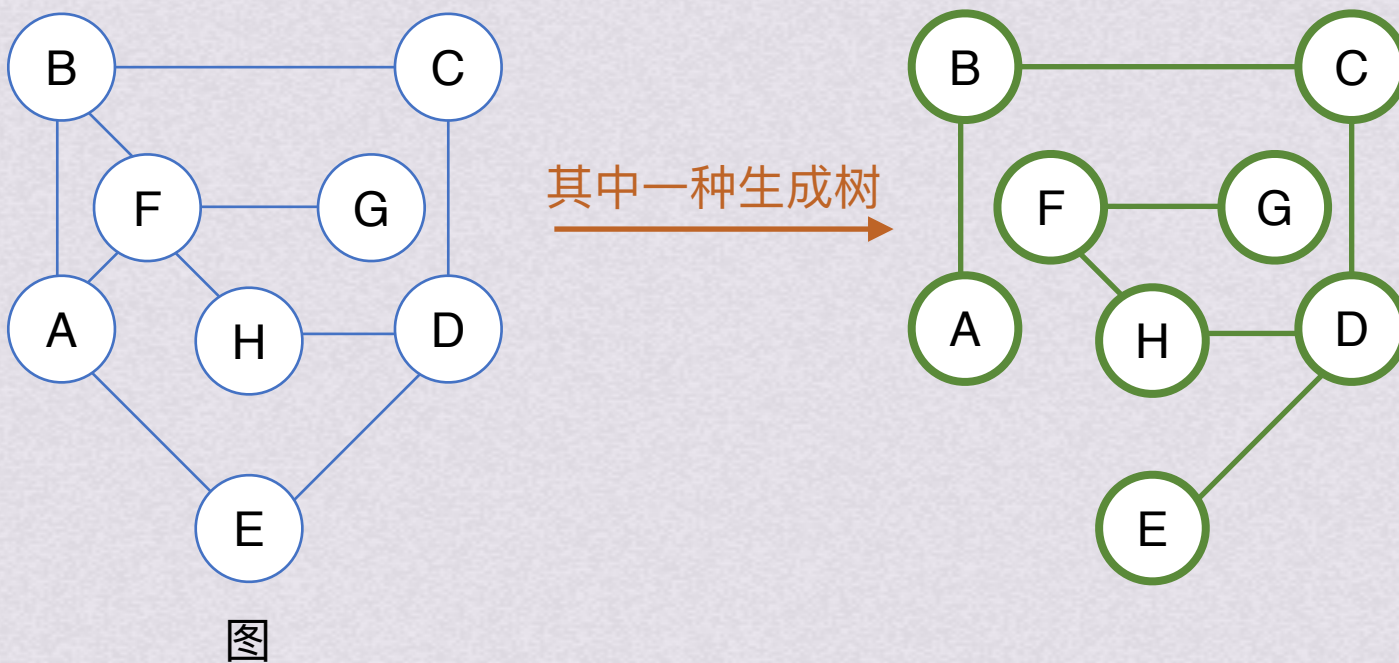
错因：仍然像刚才无向图一样只考虑把和顶点有关的弧都加上，却忘记了弧有方向

错因：没有保持强连通，要保持强连通必须去掉B顶点，加上C、D顶点



生成树

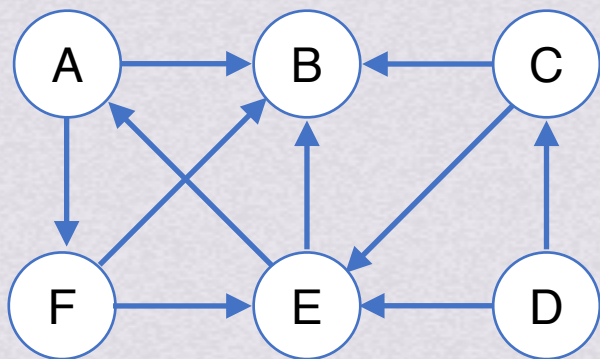
- 无向连通图中一个包含图中全部顶点的极小连通子图。
- 假设顶点数为 n ，那么生成树一定有 $n-1$ 条边。【注】反之则不对。
- 再加一条边，则该图中出现环；再删一条边，则该图不连通



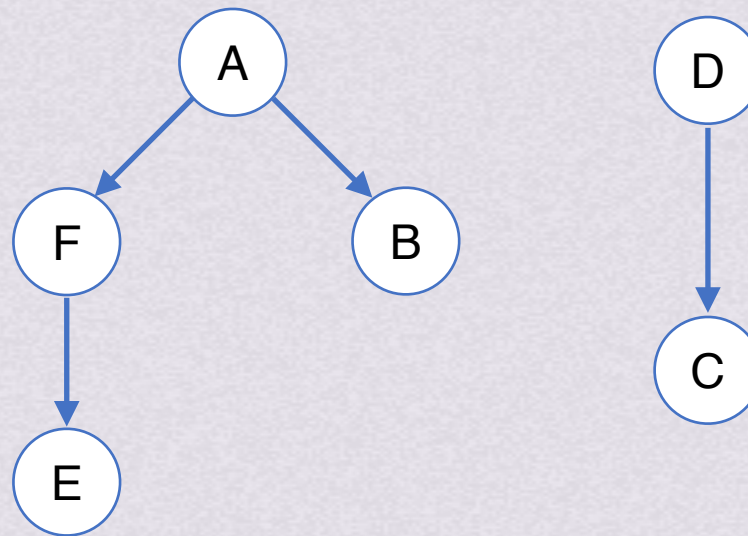


有向树和生成森林

- 有一个顶点的入度为0，其余顶点的入度均为1的有向图。
- 一个有向图的生成森林是由若干棵有向树组成，包含图中所有顶点，但只有足以让构成若干棵不相交的有向树的弧。



一个有向图

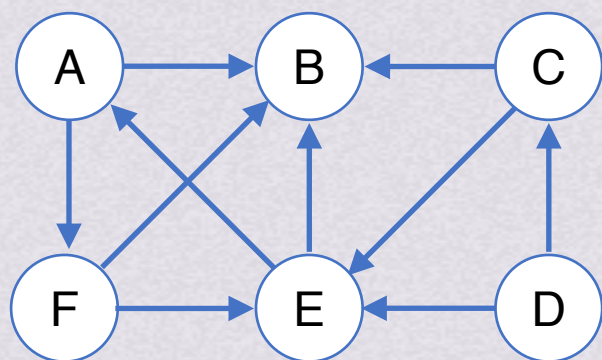


对应的生成森林

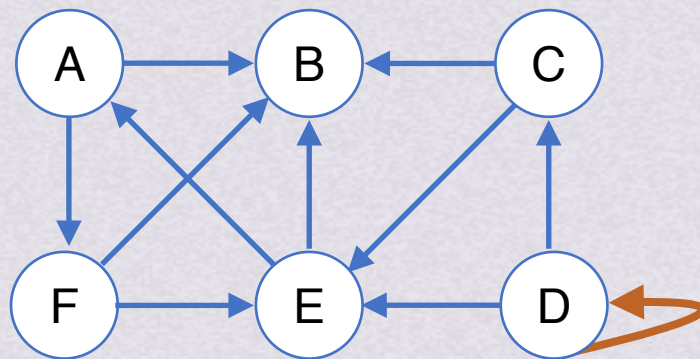


简单图

- 不存在重复的边
- 不存在顶点到自身的边。
- 408只会涉及简单图



简单图

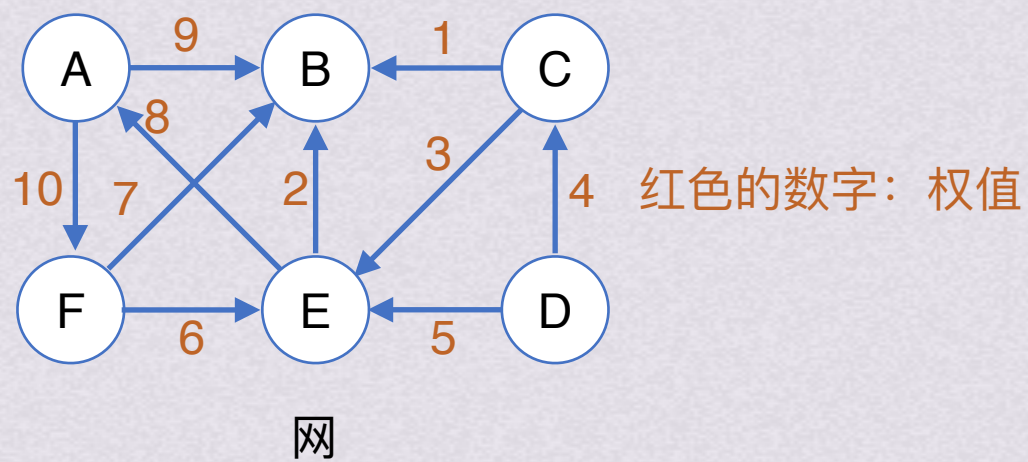


非简单图



权和网

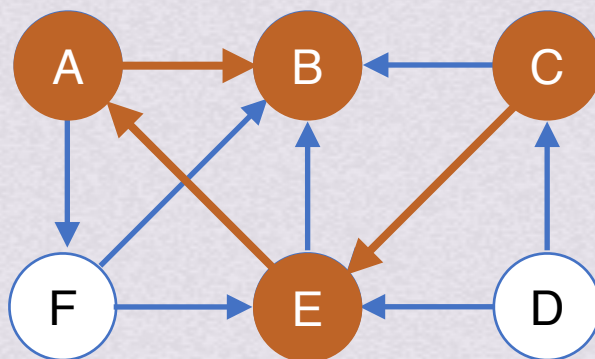
- 权：每条边/弧上具有某种含义的数值
- 网：带权的图





路径和路径长度

- 路径：顶点序列
- 路径长度：一条路径上经过的边或弧的数目。

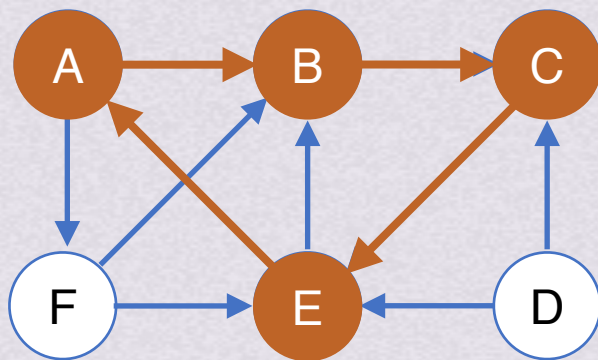


CEAB:一条路径

该路径长度为3

回路和环

- 第一个顶点和最后一个顶点相同的路径称为回路或环。
- 回路和环是相同的概念。



CEABC:一条回路/一个环



ADT&基本操作

ADT Graph{

数据对象 :V是具有相同特性的数据元素的集合, 称为顶点集。

数据关系 :

$R = \{VR\}$

$VR = \{ \langle v, w \rangle \mid v, w \in V \text{ 且 } P(v, w) \}$ 表示从v到w的弧,
谓词 $P(v, w)$ 定义了弧 $\langle v, w \rangle$ 的意义或信息}

基本操作 :

CreateGraph(&G,V,VR)

初始条件 :V是图的顶点集, VR是图中弧的集合。

操作结果 :按V和VR的定义构造图。

DestroyGraph(&G)

初始条件 :图G存在。

操作结果 :销毁图G。

LocateVex(G,u)

初始条件 :图G存在, u和G中顶点有相同特征。

操作结果 :若G中存在顶点u, 则返回该顶点在图中的位置, 否则返回其他信息。

GetVex(G,v)

初始条件 :图G存在, v是G中某个顶点。

操作结果 :返回v的值。

PutVex(&G,v,value)

初始条件 :图G存在, v是G中某个顶点。

操作结果 :对v赋值value。

FirstAdjVex(G,v)

初始条件 :图G存在, v是G中某个顶点。

操作结果 :返回 v的第一个邻接顶点。若v在G中没有邻接顶点, 则返回“空”。

NextAdjVex(G,v,w)

初始条件 :图G存在, v是G中某个顶点, w是v的邻接顶点。

操作结果 :返回v的(相对于w的)下一个邻接顶点。若w是v的最后一个邻接点, 则返回“空”。

InsertVex(&G,v)

初始条件 :图G存在, v和图中顶点有相同特征。

操作结果 :在图G中增添新顶点v。

DeleteVex(&G,v)

初始条件 :图G存在, v是G中某个顶点。

操作结果 :删除G中顶点v及其相关的弧。

InsertArc(&G,v,w)

初始条件 :图G存在, v和w是G中两个顶点。

操作结果 :在G中增添弧 $\langle v, w \rangle$, 若G是无向图, 则还增添对称弧 $\langle w, v \rangle$ 。

DeleteArc(&G,v,w)

初始条件 :图G存在, v和w是G中两个顶点。

操作结果 :在G中删除弧 $\langle v, w \rangle$, 若G是无向图, 则还删除对称弧 $\langle w, v \rangle$ 。

DFSTraverse(G)

初始条件 :图G存在。

操作结果 :对图进行深度优先遍历, 在遍历过程中对每个顶点访问一次。

BFSTraverse(G)

初始条件 :图G存在。

操作结果 :对图进行广度优先遍历, 在遍历过程中对每个顶点访问一次。

}ADT Graph